AWS Deployment Guide: Pengajuan KTA — Bappeda

> ***Versi**: 1.0 **Tanggal**: April 2026 **Stack**: Spring Boot 3.5.5 · Java 21 · PostgreSQL · Redis · Docker **Target**: AWS (ap-southeast-1 / Singapore)*

Daftar Isi

1. Overview & Architecture
2. Prerequisites
3. Phase 1: Networking (VPC, Subnet, Security)
4. Phase 2: Database (RDS PostgreSQL)
5. Phase 3: Cache (ElastiCache Redis)
6. Phase 4: Compute (EC2 + Docker)
7. Phase 5: Domain & SSL
8. Phase 6: CI/CD Pipeline
9. Phase 7: Monitoring & Logging
10. Phase 8: Backup & Disaster Recovery
11. Troubleshooting
12. Cost Summary
13. Quick Reference

---

1. Overview & Architecture

1.1 Aplikasi yang Akan Di-deploy

Komponen	Detail
Framework	Spring Boot 3.5.5 (Java 21, Gradle)
Database	PostgreSQL (via JPA + Flyway migration)
Cache	Redis (token blacklist, OTP, session)
File Storage	Cloudflare R2 (S3-compatible)
Auth	JWT + reCAPTCHA
Integrasi	SMTP Email, WhatsApp API
Container	Multi-stage Dockerfile (sudah ada)
API Port	8080 (context path: `/kta/api`)
API Docs	Swagger UI (`/kta/api/swagger-ui.html`)

1.2 Arsitektur Target

```
                    INTERNET
                       │
                       ▼
              ┌─────────────────┐
              │   Route 53      │  ← DNS: kta.bappeda.go.id
```

```
                      │   (DNS)        │     mengarah ke Elastic IP
                      └────────────────┘
                              │
                              ▼

                     ┌──────────────────┐
                     │ Internet Gateway│  ← Gerbang masuk ke VPC
                     │    (igw-xxx)    │
                     └──────────────────┘
                              │
                              │
┌─────────────────────────────┼──────────────────────────────────┐
│                             │                                  │
│  VPC: 10.0.0.0/16          │         Region: ap-southeast-1     │
│  ═══════════════            │          (Singapore)               │
│                             │                                  │
│  ┌ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─┼─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ┐  │
│    PUBLIC SUBNET            │                                  │
│  │ 10.0.1.0/24             │            AZ: ap-southeast-1a │  │
│    (Bisa diakses            │                                  │
│  │  dari internet)          ▼                               │  │
│                 ┌──────────────────────────┐                    │
│  │              │ EC2 Instance             │                 │  │
│                 │ ┌──────────────────────┐ │                    │
│  │              │ │    Elastic IP        │ │  ← IP statis    │  │
│                 │ │    (13.x.x.x)       │ │    tidak berubah     │
│  │              │ └──────────────────────┘ │                 │  │
│                 │                          │                    │
│  │              │ ┌──────────────────────┐ │                 │  │
│                 │ │  Nginx Container     │ │                    │
│  │              │ │  :443 (HTTPS)        │ │  ← SSL/TLS      │  │
│                 │ │  :80 → redirect 443  │ │    termination       │
│  │              │ └──────────────────────┘ │                 │  │
│                 │            │             │                    │
│  │              │ ┌──────────▼───────────┐ │                 │  │
│                 │ │  Spring Boot App     │ │                    │
│  │              │ │  pengajuan_kta       │ │  ← Aplikasi     │  │
│                 │ │  :8080 (/kta/api)    │ │    utama             │
│  │              │ └──────────────────────┘ │                 │  │
│                 │                          │                    │
│  │              │ Security Group: sg-app   │                 │  │
│                 │ IN:  443, 80, 22(limited)│                    │
│  │              │ OUT: all                 │                 │  │
│                 └──────────────────────────┘                    │
│  └ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─┼─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ┘  │
│                             │                                  │
│         (Traffic internal   │  via private IP 10.0.x.x)        │
│                             │                                  │
│  ┌ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─┼─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ┐  │
│    PRIVATE SUBNET A         │                                  │
│  │ 10.0.10.0/24            │            AZ: ap-southeast-1a│  │
│    (TIDAK bisa diakses      │                                  │
│  │  dari internet)          │                               │  │
│                             │
│                  ┌──────────┼──────────┐                        │
```

```
│  │              │                    │                    │  │
│          ┌────────▼────────┐   ┌────────▼────────┐              │
│  │       │ RDS PostgreSQL │   │ ElastiCache    │            │  │
│          │ db.t3.micro    │   │ Redis          │              │
│  │       │ :5432          │   │ cache.t3.micro │            │  │
│          │                │   │ :6379          │              │
│  │       │ sg-database    │   │ sg-cache       │            │  │
│          │ IN: 5432       │   │ IN: 6379       │              │
│  │       │  from sg-app   │   │  from sg-app   │            │  │
│          └────────────────┘   └────────────────┘              │
│  └ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─┘  │
│                                                               │
│  ┌ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─┐  │
│    PRIVATE SUBNET B                                           │
│  │ 10.0.11.0/24                        AZ: ap-southeast-1b│  │
│    (Wajib ada untuk RDS                                       │
│  │  Subnet Group, minimal 2 AZ)                            │  │
│  └ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─┘  │
│                                                               │
└───────────────────────────────────────────────────────────────┘
        │                           │
        ▼                           ▼
┌─────────────────┐          ┌─────────────────┐
│ Cloudflare R2   │          │ External APIs   │
│ (File Storage)  │          │ • SMTP Mail     │
│                 │          │ • WhatsApp API  │
│                 │          │ • reCAPTCHA     │
└─────────────────┘          └─────────────────┘
```

1.3 Kenapa Arsitektur Ini?

Keputusan	Alasan
EC2 (bukan ECS Fargate)	User kecil, tidak perlu ALB, lebih hemat
Nginx di Docker (bukan ALB)	Hemat ~$18/bulan, cukup untuk traffic kecil
RDS Managed (bukan self-hosted)	Automated backup, patching, encryption by default
ElastiCache Managed	Tidak perlu manage Redis, auto failover ready
Private Subnet untuk DB/Redis	Database tidak bisa diakses dari internet
Elastic IP	IP publik statis, tidak berubah saat restart
Tanpa NAT Gateway	Hemat ~$32/bulan, DB/Redis tidak perlu internet

1.4 Penjelasan Konsep (untuk Beginner)

Istilah	Analogi Sederhana
VPC	Komplek perumahan tertutup (area aman kamu di AWS)
Subnet	Blok-blok rumah dalam komplek

Public Subnet	Blok yang menghadap jalan raya (bisa diakses orang luar)
Private Subnet	Blok di dalam komplek (hanya penghuni yang bisa masuk)
Internet Gateway	Gerbang utama komplek ke jalan raya
Security Group	Satpam di setiap rumah (cek siapa yang boleh masuk per port)
Network ACL	Satpam di gerbang blok (filter per subnet)
Route Table	Papan petunjuk arah di setiap blok
Elastic IP	Nomor rumah tetap (tidak berubah walau pindah rumah)
EC2	Rumah/server tempat aplikasi berjalan
RDS	Gudang data (database) yang dijaga AWS
ElastiCache	Meja resepsionis (cache cepat untuk data sering diakses)

---

2. Prerequisites

2.1 Checklist Sebelum Mulai

- [ ] AWS Account aktif (https://aws.amazon.com)
- [ ] AWS CLI terinstall di laptop
- [ ] SSH client (terminal macOS/Linux sudah built-in)
- [ ] Git terinstall
- [ ] Domain name (opsional, bisa pakai IP dulu)
- [ ] Source code `pengajuan_kta` (sudah ada)
- [ ] Dockerfile (sudah ada)

2.2 Install AWS CLI

macOS:

```
brew install awscli
```

Linux:

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
```

Verifikasi:

```
aws --version
# Output: aws-cli/2.x.x ...
```

2.3 Konfigurasi AWS CLI

Cara mendapatkan Access Key:

1. Login ke AWS Console
2. Klik nama kamu di pojok kanan atas → **Security credentials**
3. Atau buka: IAM → Users → Pilih user → Security credentials tab
4. Klik **Create access key** → pilih "Command Line Interface (CLI)"
5. Simpan Access Key ID dan Secret Access Key

Jalankan konfigurasi:

```
aws configure
```

```
AWS Access Key ID [None]: AKIAXXXXXXXXXXXXXXXX
AWS Secret Access Key [None]: xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
Default region name [None]: ap-southeast-1
Default output format [None]: json
```

Verifikasi koneksi:

```
aws sts get-caller-identity
```

Harus menampilkan Account ID dan ARN kamu.

---

3. Phase 1: Networking

Overview Phase Ini

```
Yang akan dibuat:
  □ 3.1  VPC
  □ 3.2  Internet Gateway
  □ 3.3  Public Subnet (10.0.1.0/24)
  □ 3.4  Private Subnet A (10.0.10.0/24) — AZ-1a
  □ 3.5  Private Subnet B (10.0.11.0/24) — AZ-1b
  □ 3.6  Route Table (Public)
  □ 3.7  Route Table (Private)
  □ 3.8  Security Groups (3 buah)
  □ 3.9  Network ACL
```

3.1 Buat VPC

Apa itu? VPC (Virtual Private Cloud) = jaringan virtual terisolasi milik kamu di AWS. Semua resource akan hidup di dalam VPC ini.

Via Console:

1. Buka **VPC** → **Your VPCs** → **Create VPC**
2. Isi:
   - **Name tag**: `kta-vpc`
   - **IPv4 CIDR block**: `10.0.0.0/16`
   - **IPv6 CIDR block**: No IPv6

- **Tenancy**: Default

3. Klik **Create VPC**

Via CLI:

```
# Buat VPC
aws ec2 create-vpc \
  --cidr-block 10.0.0.0/16 \
  --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=kta-vpc}]' \
  --query 'Vpc.VpcId' --output text

# Catat VPC_ID yang muncul (vpc-xxxxxxxxx)

# Enable DNS hostnames (WAJIB untuk RDS)
aws ec2 modify-vpc-attribute \
  --vpc-id vpc-XXXXX \
  --enable-dns-hostnames '{"Value":true}'

aws ec2 modify-vpc-attribute \
  --vpc-id vpc-XXXXX \
  --enable-dns-support '{"Value":true}'
```

> ***Penjelasan CIDR `10.0.0.0/16`:***
>
> - *`/16` berarti 16 bit pertama (10.0) tidak berubah*
> - *Tersedia 65.536 IP: `10.0.0.0` sampai `10.0.255.255`*
> - *Lebih dari cukup untuk kebutuhan kita*

3.2 Buat Internet Gateway (IGW)

Apa itu? Gerbang yang menghubungkan VPC ke internet. Tanpa IGW, tidak ada resource yang bisa diakses dari luar.

Via Console:

1. **VPC** → **Internet Gateways** → **Create Internet Gateway**
2. **Name tag**: `kta-igw`
3. Setelah dibuat, klik **Actions** → **Attach to VPC** → pilih `kta-vpc`

Via CLI:

```
# Buat IGW
aws ec2 create-internet-gateway \
  --tag-specifications 'ResourceType=internet-gateway,Tags=[{Key=Name,Value=kta-
igw}]' \
  --query 'InternetGateway.InternetGatewayId' --output text

# Attach ke VPC
aws ec2 attach-internet-gateway \
  --internet-gateway-id igw-XXXXX \
  --vpc-id vpc-XXXXX
```

3.3 Buat Public Subnet

Apa itu? Subnet yang bisa diakses dari internet. EC2 (server app) akan tinggal di sini.

Via Console:

1. **VPC** → **Subnets** → **Create Subnet**
2. Isi:
   - **VPC**: `kta-vpc`
   - **Name**: `kta-public-1a`
   - **Availability Zone**: `ap-southeast-1a`
   - **IPv4 CIDR**: `10.0.1.0/24`
3. Setelah dibuat, pilih subnet → **Actions** → **Edit subnet settings**
4. Centang **Enable auto-assign public IPv4 address** → Save

> ***Penjelasan `10.0.1.0/24`:*** *256 IP address (`10.0.1.0` - `10.0.1.255`). Cukup untuk kebutuhan kita yang hanya 1 EC2.*

3.4 Buat Private Subnet A

Apa itu? Subnet yang TIDAK bisa diakses dari internet. Database dan Redis akan tinggal di sini agar aman.

Via Console:

1. **VPC** → **Subnets** → **Create Subnet**
2. Isi:
   - **VPC**: `kta-vpc`
   - **Name**: `kta-private-1a`
   - **Availability Zone**: `ap-southeast-1a`
   - **IPv4 CIDR**: `10.0.10.0/24`

3.5 Buat Private Subnet B

Kenapa perlu 2 Private Subnet? AWS RDS **mewajibkan** minimal 2 subnet di Availability Zone yang **berbeda** untuk membuat DB Subnet Group. Ini syarat dari AWS, meskipun kita hanya deploy 1 instance RDS.

Via Console:

1. **VPC** → **Subnets** → **Create Subnet**
2. Isi:
   - **VPC**: `kta-vpc`
   - **Name**: `kta-private-1b`
   - **Availability Zone**: `ap-southeast-1b` ← **BEDA AZ dari yang sebelumnya!**
   - **IPv4 CIDR**: `10.0.11.0/24`

Hasil Setelah Step 3.1 - 3.5

```
┌──────────────────────────────────────────────────────┐
│ VPC: kta-vpc (10.0.0.0/16)                           │
│                                                       │
│  ┌──────────────────────────┐                         │
│  │ kta-public-1a            │   AZ: 1a              │
│  │ 10.0.1.0/24              │   (EC2 di sini)       │
```

```
│   └──────────────────────────┘                        │
│                                                       │
│   ┌──────────────────────────┐                        │
│   │ kta-private-1a           │   AZ: 1a              │
│   │ 10.0.10.0/24             │   (RDS + Redis)       │
│   └──────────────────────────┘                        │
│                                                       │
│   ┌──────────────────────────┐                        │
│   │ kta-private-1b           │   AZ: 1b              │
│   │ 10.0.11.0/24             │   (RDS Subnet Group)  │
│   └──────────────────────────┘                        │
│                                                       │
│  Internet Gateway: kta-igw (attached)                 │
└──────────────────────────────────────────────────────┘
```

3.6 Buat Route Table — Public

Apa itu? Route Table menentukan ke mana traffic jaringan diarahkan. Public Route Table akan mengarahkan traffic internet ke IGW.

Via Console:

1. **VPC** → **Route Tables** → **Create Route Table**
   - **Name**: `kta-rt-public`
   - **VPC**: `kta-vpc`
2. Pilih route table → tab **Routes** → **Edit routes** → **Add route**:
   - **Destination**: `0.0.0.0/0`
   - **Target**: Internet Gateway → `kta-igw`
   - Klik **Save changes**
3. Tab **Subnet associations** → **Edit subnet associations** → centang `kta-public-1a` → Save

> ***Artinya:*** *"Semua traffic yang tujuannya ke internet (`0.0.0.0/0`), lewatkan melalui Internet Gateway."*

3.7 Buat Route Table — Private

Via Console:

1. **VPC** → **Route Tables** → **Create Route Table**
   - **Name**: `kta-rt-private`
   - **VPC**: `kta-vpc`
2. **JANGAN** tambah route ke internet. Biarkan hanya ada route `local`:
   - `10.0.0.0/16` → `local` (sudah otomatis ada)
3. Tab **Subnet associations** → **Edit** → centang `kta-private-1a` dan `kta-private-1b` → Save

> ***Artinya:*** *"Resource di private subnet hanya bisa komunikasi di dalam VPC. Tidak bisa ke internet."*

3.8 Buat Security Groups

Security Group = firewall virtual pada level instance. Mengontrol traffic masuk (inbound) dan keluar (outbound).

SG 1: `kta-sg-app` (untuk EC2)

Direction	Type	Port	Source	Keterangan
Inbound	HTTPS	443	`0.0.0.0/0`	Akses web dari mana saja
Inbound	HTTP	80	`0.0.0.0/0`	Redirect ke HTTPS
Inbound	SSH	22	`IP_KANTOR/32`	SSH hanya dari IP kantor
Outbound	All traffic	All	`0.0.0.0/0`	Akses internet keluar

Via Console:

1. **VPC** → **Security Groups** → **Create Security Group**
2. **Name**: `kta-sg-app` · **VPC**: `kta-vpc`
3. Tambah Inbound Rules sesuai tabel di atas
4. Outbound biarkan default (All traffic)

> ***PENTING:*** *Ganti* `IP_KANTOR/32` *dengan IP publik kantor/rumah kamu. Cek di https://whatismyipaddress.com. `/32` artinya hanya 1 IP itu saja yang diizinkan.*

SG 2: `kta-sg-database` (untuk RDS)

Direction	Type	Port	Source	Keterangan
Inbound	PostgreSQL	5432	`kta-sg-app`	Hanya dari EC2 app
Outbound	—	—	—	Tidak perlu

SG 3: `kta-sg-cache` (untuk ElastiCache Redis)

Direction	Type	Port	Source	Keterangan
Inbound	Custom TCP	6379	`kta-sg-app`	Hanya dari EC2 app
Outbound	—	—	—	Tidak perlu

> ***Prinsip Least Privilege:*** *Database dan Redis HANYA menerima koneksi dari Security Group `kta-sg-app`. Tidak ada akses dari internet.*

Security Visualization

```
                    INTERNET
                       │
    ┌──────────────────┼──────────────────┐
    │                  ▼                  │
    │         ┌─────────────────┐         │
    │         │   kta-sg-app    │         │
    │         │ ✅ 443 from ALL │         │
    │         │ ✅ 80  from ALL │         │
    │         │ ✅ 22  from IP  │         │
    │         └─────────────────┘         │
    │           │          │              │
    │      ┌────▼────┐  ┌──▼──────┐      │
    │      │sg-database│ │ sg-cache │      │
```

```
    │      │✅ 5432     │ │ ✅ 6379    │     │
    │      │from sg-app│ │ from       │     │
    │      │ ONLY       │ │ sg-app     │     │
    │      │            │ │ ONLY       │     │
    │      │❌ internet│ │ ❌internet│     │
    │      └────────────┘ └────────────┘     │
    └──────────────────────────────────────┘
```

3.9 Network ACL (Optional, Extra Layer)

Network ACL = firewall pada level **subnet** (bukan per instance). Ini opsional tapi menambah layer keamanan.

NACL Public Subnet

Rule	Direction	Type	Port	Source/Dest	Action
100	Inbound	HTTPS	443	`0.0.0.0/0`	ALLOW
110	Inbound	HTTP	80	`0.0.0.0/0`	ALLOW
120	Inbound	SSH	22	`IP_KANTOR/32`	ALLOW
130	Inbound	Custom TCP	1024-65535	`0.0.0.0/0`	ALLOW
*	Inbound	All	All	`0.0.0.0/0`	DENY
100	Outbound	All	All	`0.0.0.0/0`	ALLOW

> ***Port 1024-65535*** *= ephemeral ports, diperlukan untuk response traffic.*

NACL Private Subnet

Rule	Direction	Type	Port	Source/Dest	Action
100	Inbound	Custom TCP	5432	`10.0.1.0/24`	ALLOW
110	Inbound	Custom TCP	6379	`10.0.1.0/24`	ALLOW
*	Inbound	All	All	`0.0.0.0/0`	DENY
100	Outbound	Custom TCP	1024-65535	`10.0.1.0/24`	ALLOW
*	Outbound	All	All	`0.0.0.0/0`	DENY

---

4. Phase 2: Database (RDS PostgreSQL)

4.1 Buat DB Subnet Group

Apa itu? Kumpulan subnet tempat RDS boleh di-deploy. AWS mewajibkan minimal 2 subnet di AZ berbeda.

Via Console:

1. **RDS** → **Subnet groups** → **Create DB subnet group**
2. Isi:
   - **Name**: `kta-db-subnet-group`
   - **Description**: `Subnet group for KTA database`

- **VPC**: `kta-vpc`

3. Add subnets:
   - **AZ** **`ap-southeast-1a`** → pilih `kta-private-1a` (10.0.10.0/24)
   - **AZ** **`ap-southeast-1b`** → pilih `kta-private-1b` (10.0.11.0/24)
4. Klik **Create**

4.2 Buat RDS PostgreSQL Instance

Via Console:

1. **RDS** → **Databases** → **Create database**
2. Isi sebagai berikut:

Setting	Value
Creation method	Standard Create
Engine	PostgreSQL
Engine version	16.x (pilih yang terbaru)
Template	**Free Tier** ← pilih ini untuk hemat
DB instance identifier	`kta-database`
Master username	`kta_admin`
Master password	*(buat password kuat, catat!)*
Instance class	`db.t3.micro`
Storage type	gp3
Allocated storage	20 GB
Storage autoscaling	Enable (max 50 GB)
VPC	`kta-vpc`
DB subnet group	`kta-db-subnet-group`
Public access	**No** ← PENTING!
Security group	`kta-sg-database`
Database port	5432
Initial database name	`pengajuan_kta`
Backup retention	7 days
Encryption	Enable
Monitoring	Enable Enhanced Monitoring (free tier: 60s)

3. Klik **Create database** (pembuatan memakan waktu ~10 menit)

Via CLI:

```
aws rds create-db-instance \
  --db-instance-identifier kta-database \
  --db-instance-class db.t3.micro \
  --engine postgres \
  --engine-version 16.4 \
  --master-username kta_admin \
  --master-user-password "YOUR_STRONG_PASSWORD" \
  --allocated-storage 20 \
  --storage-type gp3 \
  --db-name pengajuan_kta \
  --vpc-security-group-ids sg-XXXXX \
  --db-subnet-group-name kta-db-subnet-group \
  --backup-retention-period 7 \
  --no-publicly-accessible \
  --storage-encrypted \
  --port 5432
```

4. **Setelah RDS selesai dibuat**, catat endpoint:

```
kta-database.xxxxxxxxxxxx.ap-southeast-1.rds.amazonaws.com
```

Endpoint ini yang akan dipakai di environment variable `SPRING_DATASOURCE_URL`.

---

5. Phase 3: Cache (ElastiCache Redis)

5.1 Buat Cache Subnet Group

Via Console:

1. **ElastiCache** → **Subnet groups** → **Create subnet group**
2. Isi:
   - **Name**: `kta-cache-subnet-group`
   - **VPC**: `kta-vpc`
   - **Subnets**: pilih `kta-private-1a` (10.0.10.0/24)

5.2 Buat Redis Cluster

Via Console:

1. **ElastiCache** → **Redis caches** → **Create Redis cache**
2. Isi:

Setting	Value
Cluster mode	Disabled
Name	`kta-redis`
Description	`Redis cache for KTA app`
Node type	`cache.t3.micro`
Number of replicas	0 (hemat biaya)

Subnet group	`kta-cache-subnet-group`
Security group	`kta-sg-cache`
Encryption in-transit	Enable
Encryption at-rest	Enable
Port	6379

3. Klik **Create**

Setelah selesai, catat endpoint:

```
kta-redis.xxxxxx.0001.apse1.cache.amazonaws.com:6379
```

---

6. Phase 4: Compute (EC2 + Docker)

6.1 Buat Key Pair

Key Pair digunakan untuk SSH ke EC2. Ini seperti "kunci rumah" digital.

Via Console:

1. **EC2 → Key Pairs → Create key pair**
2. Isi:
   - **Name**: `kta-keypair`
   - **Type**: RSA
   - **Format**: `.pem` (untuk macOS/Linux)
3. File `.pem` akan otomatis ter-download. **Simpan dengan aman! File ini tidak bisa di-download ulang.**

```
# Set permission (WAJIB, SSH menolak key dengan permission terlalu terbuka)
chmod 400 ~/Downloads/kta-keypair.pem

# Pindahkan ke folder yang aman
mv ~/Downloads/kta-keypair.pem ~/.ssh/kta-keypair.pem
```

6.2 Launch EC2 Instance

Via Console:

1. **EC2 → Instances → Launch instances**
2. Isi:

Setting	Value
Name	`kta-app-server`
AMI	Amazon Linux 2023 (Free Tier eligible)
Instance type	`t3.small` (2 vCPU, 2 GB RAM)

Key pair	`kta-keypair`
VPC	`kta-vpc`
Subnet	`kta-public-1a`
Auto-assign public IP	Enable
Security group	`kta-sg-app` (select existing)
Storage	30 GB gp3

3. Klik **Launch instance**

> ***Kenapa `t3.small`?*** *Spring Boot + Docker membutuhkan minimal ~1.5 GB RAM.* `t3.micro` *(1 GB) bisa ketat.* `t3.small` *(2 GB) lebih aman.*

6.3 Allocate Elastic IP

Kenapa? Public IP EC2 **berubah** setiap kali instance di-restart. Elastic IP = IP publik yang **tetap**.

Via Console:

1. **EC2** → **Elastic IPs** → **Allocate Elastic IP address** → Allocate
2. Pilih Elastic IP yang baru → **Actions** → **Associate Elastic IP address**
3. Pilih instance: `kta-app-server` → Associate

Catat Elastic IP ini (misalnya `13.212.xxx.xxx`). Ini akan jadi IP publik server kamu.

6.4 SSH ke EC2

```
ssh -i ~/.ssh/kta-keypair.pem ec2-user@ELASTIC_IP
```

Ketik `yes` saat diminta fingerprint confirmation.

6.5 Install Docker & Docker Compose di EC2

Jalankan perintah berikut setelah SSH:

```
# Update system
sudo dnf update -y

# Install Docker
sudo dnf install docker -y
sudo systemctl start docker
sudo systemctl enable docker

# Tambah user ec2-user ke group docker (agar bisa jalankan docker tanpa sudo)
sudo usermod -aG docker ec2-user

# Install Docker Compose v2
sudo mkdir -p /usr/local/lib/docker/cli-plugins
sudo curl -SL "https://github.com/docker/compose/releases/latest/download/docker-
compose-$(uname -s)-$(uname -m)" \
```

```
  -o /usr/local/lib/docker/cli-plugins/docker-compose
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose

# Install git (untuk clone repo)
sudo dnf install git -y

# PENTING: Logout dan login ulang agar group docker aktif
exit
```

```
# Login ulang
ssh -i ~/.ssh/kta-keypair.pem ec2-user@ELASTIC_IP

# Verifikasi
docker --version
docker compose version
git --version
```

6.6 Setup Aplikasi di EC2

```
# Buat directory
mkdir -p /home/ec2-user/kta-app
cd /home/ec2-user/kta-app

# Clone repository
git clone https://github.com/bappeda-dev-team/pengajuan-kta.git .
```

Buat file `.env`:

```
nano .env
```

Isi dengan (ganti value sesuai environment kamu):

```
# Database (gunakan endpoint RDS dari Phase 2)
SPRING_DATASOURCE_URL=jdbc:postgresql://kta-database.xxxxx.ap-southeast-
1.rds.amazonaws.com:5432/pengajuan_kta
SPRING_DATASOURCE_USERNAME=kta_admin
SPRING_DATASOURCE_PASSWORD=password_rds_kamu

# Redis (gunakan endpoint ElastiCache dari Phase 3)
SPRING_DATA_REDIS_HOST=kta-redis.xxxxx.0001.apse1.cache.amazonaws.com
SPRING_DATA_REDIS_PORT=6379

# JWT
JWT_SECRET=your_very_long_random_jwt_secret_at_least_64_characters
JWT_EXPIRATION_MILLIS=86400000

# Mail
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
```

```
MAIL_USERNAME=your_email@gmail.com
MAIL_PASSWORD=your_app_password

# Cloudflare R2
CLOUDFLARE_BUCKET=your_bucket_name
CLOUDFLARE_ENDPOINT=https://xxxxx.r2.cloudflarestorage.com
CLOUDFLARE_ACCESS_KEY=your_access_key
CLOUDFLARE_SECRET_KEY=your_secret_key
CLOUDFLARE_BASE_URL=https://your-public-r2-url.com

# WhatsApp
WHATSAPP_API_URL=https://api.whatsapp.com/xxx
WHATSAPP_API_TOKEN=your_token

# reCAPTCHA
RECAPTCHA_SITE_KEY=your_site_key
RECAPTCHA_SECRET_KEY=your_secret_key
```

Buat Nginx config:

```
mkdir -p nginx/conf.d nginx/ssl
```

File `nginx/conf.d/default.conf` sudah disediakan di repository (lihat bagian file yang dibuat di bawah).

6.7 Build & Jalankan

```
cd /home/ec2-user/kta-app

# Build dan jalankan semua service
docker compose up -d --build

# Cek status (semua harus "Up")
docker compose ps

# Cek log aplikasi (Ctrl+C untuk keluar)
docker compose logs -f app

# Test endpoint
curl http://localhost:8080/kta/api/v3/api-docs
```

Hasil Setelah Phase 4

```
EC2 Instance:
  ┌──────────────────────────────────────┐
  │  Nginx (:443) ── SSL ──┐             │
  │  Nginx (:80)  ── redirect 443        │
  │                          │           │
  │                     ┌────▼────┐      │
  │                     │ Spring Boot│   │
  │                     │ :8080      │   │
```

```
│                    │ /kta/api/* │  │
│                    └────────────┘  │
│                          │         │
│              ┌───────────┼─────┐   │
│              ▼           ▼     │   │
│          RDS:5432    Redis:6379 │  │
│         (private)   (private)  │  │
└──────────────────────────────────┘
```

---

7. Phase 5: Domain & SSL

7.1 Setup DNS (Route 53)

Jika menggunakan Route 53:

1. **Route 53** → **Hosted zones** → **Create hosted zone**
   - **Domain**: `bappeda.go.id` (atau domain kamu)
2. **Create record**:
   - **Name**: `kta` (menjadi `kta.bappeda.go.id`)
   - **Type**: A
   - **Value**: *(Elastic IP EC2 kamu)*
   - **TTL**: 300

Jika domain di-manage di luar AWS:

Tambahkan A record di DNS provider kamu:

- **Host**: `kta`
- **Type**: A
- **Value**: *(Elastic IP kamu)*

7.2 Setup SSL (Let's Encrypt)

SSL membuat koneksi terenkripsi (HTTPS). Gratis dengan Let's Encrypt.

```
# SSH ke EC2
ssh -i ~/.ssh/kta-keypair.pem ec2-user@ELASTIC_IP

# Install Certbot
sudo dnf install certbot -y

# Stop Nginx dulu (port 80 harus free untuk verifikasi)
cd /home/ec2-user/kta-app
docker compose stop nginx

# Generate SSL certificate
sudo certbot certonly --standalone \
  -d kta.bappeda.go.id \
  --email admin@bappeda.go.id \
  --agree-tos \
  --no-eff-email
```

```
# Copy certificate ke folder nginx
sudo cp /etc/letsencrypt/live/kta.bappeda.go.id/fullchain.pem nginx/ssl/
sudo cp /etc/letsencrypt/live/kta.bappeda.go.id/privkey.pem nginx/ssl/
sudo chown ec2-user:ec2-user nginx/ssl/*.pem

# Jalankan Nginx lagi
docker compose up -d nginx

# Test HTTPS
curl -I https://kta.bappeda.go.id/kta/api/v3/api-docs
```

7.3 Auto-Renew SSL

SSL Let's Encrypt berlaku 90 hari. Setup auto-renew:

```
# Buat script renewal
cat << 'SCRIPT' > /home/ec2-user/renew-ssl.sh
#!/bin/bash
set -e
cd /home/ec2-user/kta-app
docker compose stop nginx
sudo certbot renew --quiet
sudo cp /etc/letsencrypt/live/kta.bappeda.go.id/fullchain.pem nginx/ssl/
sudo cp /etc/letsencrypt/live/kta.bappeda.go.id/privkey.pem nginx/ssl/
sudo chown ec2-user:ec2-user nginx/ssl/*.pem
docker compose up -d nginx
echo "[$(date)] SSL renewed successfully" >> /home/ec2-user/ssl-renew.log
SCRIPT

chmod +x /home/ec2-user/renew-ssl.sh

# Jalankan otomatis setiap tanggal 1 jam 3 pagi
(crontab -l 2>/dev/null; echo "0 3 1 * * /home/ec2-user/renew-ssl.sh") | crontab -
```

---

8. Phase 6: CI/CD Pipeline

8.1 Deployment Flow

```
Developer                GitHub                   AWS EC2
   |                        |                        |
   |  git push main         |                        |
   | ———————————————————————►|                        |
   |                        |                        |
   |                        |  1. Checkout code      |
   |                        |  2. Vulnerability scan |
   |                        |  3. Build Docker image |
   |                        |  4. Push to GHCR       |
   |                        |                        |
   |                        |  5. SSH to EC2         |
   |                        | ———————————————————————►|
```

```
    |                        |                        |  6. Pull image
    |                        |                        |  7. Restart app
    |                        |                        |
    |                        |  Deploy complete ✅    |
    |                        | ◀──────────────────────|
    |                        |                        |
```

8.2 GitHub Actions: Deploy Stage

Tambahkan deploy job ke workflow yang sudah ada, atau buat workflow terpisah.

File: `.github/workflows/deploy.yml` (sudah disediakan di repository, lihat di bawah)

8.3 Setup GitHub Secrets

Buka repository di GitHub → **Settings** → **Secrets and variables** → **Actions** → **New repository secret**

Secret Name	Value	Cara Mendapatkan
`EC2_HOST`	`13.212.xxx.xxx`	Elastic IP dari Phase 4
`EC2_SSH_KEY`	*(isi dari file kta-keypair.pem)*	`cat ~/.ssh/kta-keypair.pem` lalu paste seluruh isi

8.4 Cara Deploy Manual (tanpa CI/CD)

Jika CI/CD belum siap, bisa deploy manual:

```
# SSH ke EC2
ssh -i ~/.ssh/kta-keypair.pem ec2-user@ELASTIC_IP

# Pull code terbaru
cd /home/ec2-user/kta-app
git pull origin main

# Rebuild dan restart
docker compose down
docker compose up -d --build

# Bersihkan image lama
docker image prune -f

# Verifikasi
docker compose ps
docker compose logs -f app
```

---

9. Phase 7: Monitoring & Logging

9.1 CloudWatch Alarms

Setup alarm agar kamu dikirim email jika ada masalah.

Via Console:

1. **CloudWatch → Alarms → Create alarm**

Alarm	Metric	Threshold	Aksi
CPU Tinggi	EC2 CPUUtilization	> 80% selama 5 menit	Email via SNS
Disk Penuh	EBS VolumeWriteBytes	> 85% used	Email via SNS
RDS CPU	RDS CPUUtilization	> 80% selama 5 menit	Email via SNS
RDS Storage	RDS FreeStorageSpace	< 2 GB	Email via SNS

Setup SNS untuk notifikasi email:

1. **SNS → Topics → Create topic**
   - **Type**: Standard
   - **Name**: `kta-alerts`
2. **Create subscription**:
   - **Protocol**: Email
   - **Endpoint**: `admin@bappeda.go.id`
3. Konfirmasi email yang dikirim AWS

9.2 Application Log Management

```
# Lihat log real-time
docker compose logs -f app
docker compose logs -f nginx

# Lihat 100 baris terakhir
docker compose logs --tail=100 app

# Setup log rotation (agar disk tidak penuh)
sudo tee /etc/logrotate.d/docker-containers > /dev/null << 'EOF'
/var/lib/docker/containers/*/*.log {
  rotate 7
  daily
  compress
  size=10M
  missingok
  delaycompress
  copytruncate
}
EOF
```

9.3 Health Check Script

```
# Buat script health check
cat << 'SCRIPT' > /home/ec2-user/healthcheck.sh
#!/bin/bash

STATUS=$(curl -s -o /dev/null -w "%{http_code}"
```

```
http://localhost:8080/kta/api/v3/api-docs)

if [ "$STATUS" != "200" ]; then
  echo "[$(date)] App DOWN (HTTP $STATUS) - Restarting..." >> /home/ec2-
user/healthcheck.log
  cd /home/ec2-user/kta-app
  docker compose restart app
else
  echo "[$(date)] App OK (HTTP $STATUS)" >> /home/ec2-user/healthcheck.log
fi
SCRIPT

chmod +x /home/ec2-user/healthcheck.sh

# Jalankan setiap 5 menit
(crontab -l 2>/dev/null; echo "*/5 * * * * /home/ec2-user/healthcheck.sh") | crontab
-
```

---

10. Phase 8: Backup & Disaster Recovery

10.1 Backup Strategy

```
Komponen           | Metode               | Frekuensi     | Retensi
-------------------|----------------------|---------------|----------
RDS PostgreSQL     | Automated Backup     | Daily         | 7 hari
                   | Manual Snapshot      | Sebelum       | Permanen
                   |                      | deploy besar  |
-------------------|----------------------|---------------|----------
EC2 / EBS Volume   | EBS Snapshot         | Weekly        | 4 minggu
                   | AMI Backup           | Setelah       | Permanen
                   |                      | setup awal    |
-------------------|----------------------|---------------|----------
Application Code   | Git repository       | Every push    | Unlimited
-------------------|----------------------|---------------|----------
Config / Secrets   | .env backup          | Setiap ada    | Encrypted
                   | (encrypted)          | perubahan     |
```

10.2 Automated EBS Snapshot

```
cat << 'SCRIPT' > /home/ec2-user/backup-ebs.sh
#!/bin/bash

INSTANCE_ID=$(curl -s http://169.254.169.254/latest/meta-data/instance-id)
VOLUME_ID=$(aws ec2 describe-instances \
  --instance-ids "$INSTANCE_ID" \
  --query 'Reservations[0].Instances[0].BlockDeviceMappings[0].Ebs.VolumeId' \
  --output text)

SNAPSHOT_ID=$(aws ec2 create-snapshot \
```

```
  --volume-id "$VOLUME_ID" \
  --description "KTA App Weekly Backup $(date +%Y-%m-%d)" \
  --query 'SnapshotId' --output text)

echo "[$(date)] Snapshot created: $SNAPSHOT_ID" >> /home/ec2-user/backup.log

# Hapus snapshot yang lebih dari 30 hari
aws ec2 describe-snapshots \
  --owner-ids self \
  --query "Snapshots[?StartTime<='$(date -d '30 days ago' +%Y-%m-%d)'].SnapshotId" \
  --output text | tr '\t' '\n' | while read sid; do
    aws ec2 delete-snapshot --snapshot-id "$sid"
    echo "[$(date)] Deleted old snapshot: $sid" >> /home/ec2-user/backup.log
done
SCRIPT

chmod +x /home/ec2-user/backup-ebs.sh

# Jalankan setiap Minggu jam 2 pagi
(crontab -l 2>/dev/null; echo "0 2 * * 0 /home/ec2-user/backup-ebs.sh") | crontab -
```

10.3 RDS Manual Snapshot (Sebelum Deploy Besar)

```
aws rds create-db-snapshot \
  --db-instance-identifier kta-database \
  --db-snapshot-identifier "kta-pre-deploy-$(date +%Y%m%d)"
```

10.4 Recovery Procedures

Jika EC2 mati:

```
# Restore dari EBS Snapshot
# 1. Buat volume baru dari snapshot
aws ec2 create-volume --snapshot-id snap-XXXXX --availability-zone ap-southeast-1a

# 2. Detach volume lama, attach volume baru ke EC2
# 3. Atau: Launch EC2 baru dengan AMI backup
```

Jika Database corrupt:

```
# Restore RDS dari automated backup (point-in-time)
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier kta-database \
  --target-db-instance-identifier kta-database-restored \
  --restore-time "2026-04-01T12:00:00Z"
```

10.5 Recovery Objectives

Metric	Target	Keterangan

RTO (Recovery Time Objective)	< 1 jam	Waktu dari gagal → kembali online
RPO (Recovery Point Objective)	< 24 jam	Maksimal data yang hilang

---

11. Troubleshooting

Masalah Umum & Solusi

Masalah	Kemungkinan Penyebab	Solusi
Tidak bisa akses web	Security Group port 80/443 belum open	Cek inbound rules `kta-sg-app`
	Nginx tidak running	`docker compose ps` → restart nginx
	Elastic IP belum di-associate	Cek EC2 → Elastic IPs
App tidak connect ke RDS	SG database tidak izinkan dari sg-app	Cek `kta-sg-database` inbound
	Endpoint RDS salah di `.env`	Cek RDS → Connectivity & security
	RDS belum ready (status != Available)	Tunggu sampai status "Available"
App tidak connect ke Redis	SG cache tidak izinkan dari sg-app	Cek `kta-sg-cache` inbound
	Endpoint Redis salah di `.env`	Cek ElastiCache → cluster details
SSH timeout	Port 22 tidak open untuk IP kamu	Update SG, tambah IP kamu
	IP kamu berubah (ISP dynamic)	Update SG dengan IP baru
Disk penuh	Docker images/logs menumpuk	`docker system prune -a`
Out of memory	Instance terlalu kecil	Upgrade ke `t3.medium` (4 GB)
		Atau tambah swap space

Perintah Debug Berguna

```
# Cek status semua container
docker compose ps

# Lihat log error aplikasi
docker compose logs --tail=200 app | grep -i error

# Cek penggunaan disk
df -h
```

```
# Cek penggunaan memory
free -m

# Cek koneksi ke RDS dari dalam container
docker exec -it pengajuan-kta bash
# Lalu di dalam container:
curl -v telnet://RDS_ENDPOINT:5432

# Cek koneksi ke Redis dari dalam container
docker exec -it pengajuan-kta bash
curl -v telnet://REDIS_ENDPOINT:6379

# Restart aplikasi saja (tanpa rebuild)
docker compose restart app

# Full rebuild dari awal
docker compose down
docker compose up -d --build

# Bersihkan semua (HATI-HATI: hapus semua container & image)
docker system prune -a
```

Menambah Swap Space (jika memory kurang)

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile swap swap defaults 0 0' | sudo tee -a /etc/fstab
```

---

12. Cost Summary

Estimasi Biaya Bulanan

Service	Spec	Biaya/bulan	Catatan
EC2	t3.small (2 vCPU, 2 GB)	~$15.26	On-demand
EBS	30 GB gp3	~$2.40	
Elastic IP	1 IP	~$3.60	
RDS PostgreSQL	db.t3.micro	~$12.41	Free tier: $0 tahun 1
ElastiCache Redis	cache.t3.micro	~$12.24	
Route 53	1 hosted zone	~$0.50	
Data Transfer	~5 GB/bulan	~$0.45	

CloudWatch	Basic + 1 alarm	~$3.00	
TOTAL		**~$50/bulan**	**~Rp 800.000**

Tips Menghemat Biaya

1. **Reserved Instance** (1 tahun commit): Hemat ~30% untuk EC2 dan RDS
2. **Free Tier** RDS: Tahun pertama gratis untuk `db.t3.micro`
3. **Spot Instance** untuk development: Hemat sampai 70% (tidak untuk production)
4. **Matikan environment dev** saat tidak dipakai
5. **Monitor billing**: AWS Console → Billing → Cost Explorer

Alternatif Paling Hemat: All-in-One EC2

Jika ingin lebih hemat, jalankan PostgreSQL dan Redis sebagai container Docker di EC2 yang sama:

```
EC2 t3.small ($15) + EBS ($2.40) + EIP ($3.60) = ~$21/bulan
```

Trade-off: Jika EC2 mati, database juga mati. Harus rajin backup.

---

13. Quick Reference

Semua Endpoint

```
Application  : https://kta.bappeda.go.id/kta/api
Swagger UI   : https://kta.bappeda.go.id/kta/api/swagger-ui.html
API Docs     : https://kta.bappeda.go.id/kta/api/v3/api-docs
```

SSH

```
ssh -i ~/.ssh/kta-keypair.pem ec2-user@ELASTIC_IP
```

Docker Commands

```
cd /home/ec2-user/kta-app
docker compose ps              # Status
docker compose logs -f app     # Log aplikasi
docker compose restart app     # Restart
docker compose down            # Stop semua
docker compose up -d --build   # Rebuild & start
```

Resource Identifiers

Resource	Name/ID
VPC	`kta-vpc`
Public Subnet	`kta-public-1a` (10.0.1.0/24)
Private Subnet A	`kta-private-1a` (10.0.10.0/24)

Private Subnet B	`kta-private-1b` (10.0.11.0/24)
Internet Gateway	`kta-igw`
Route Table Public	`kta-rt-public`
Route Table Private	`kta-rt-private`
SG App	`kta-sg-app`
SG Database	`kta-sg-database`
SG Cache	`kta-sg-cache`
EC2	`kta-app-server`
Key Pair	`kta-keypair`
RDS	`kta-database`
ElastiCache	`kta-redis`

Guide ini dibuat untuk deployment aplikasi Pengajuan KTA — Bappeda ke AWS. Terakhir diperbarui: April 2026